

Manual

Purpose

The tool addresses Lab 3: Configuration Performance Tuning. It compares a Random Search baseline against RF-SMBO, a random-forest-guided sequential optimiser that operates under a fixed measurement budget.

Expected Input

The experiment runner expects CSV datasets in `data/`. Each file should contain one valid configuration per row, configuration options in the feature columns, and a final performance column containing the runtime to minimise.

The included Lab 3 datasets already follow this structure and can be used directly without reformatting.

How to Run the Tool

Final experiment command

```
python run_experiments.py \  
  --systems 7z LLVM x264 \  
  --budgets 10 20 30 40 50 \  
  --repeats 30 \  
  --seed 20260424 \  
  --initial-design 6 \  
  --candidate-pool 4096 \  
  --kappa 0.5 \  
  --explore-prob 0.1 \  
  --output-dir outputs/final_run
```

Rebuild report PDFs

```
python build_submission_docs.py --artifact-link "https://github.com/zhaibuqun-cell/rf-smbo-configuration-tuning"
```

Output Files

- `raw_results.csv`: one row per measured step, including system, algorithm, repeat, seed, sampled index, and current best value.
- `summary_results.csv`: aggregated means and standard deviations by system, algorithm, and budget.
- `statistics.csv`: one-sided paired Wilcoxon tests and RF-SMBO win rates.
- `figure_1_convergence.png` and `figure_2_per_system.png`: report-ready matplotlib figures.

How to Interpret Results

- Lower runtime is better.
- Lower normalised gap is better.
- The win rate is the percentage of paired runs where RF-SMBO ends with a smaller final normalised gap than Random Search.
- Small p-values provide evidence that RF-SMBO beats the baseline on that system and budget.